# A Hybrid Offline-Online Routing Framework for Battery-Constrained Drone Delivery in Wireless Rechargeable Sensor Networks

CS5100 Foundations of Artificial Intelligence — Capstone Project

Northeastern University, Khoury College of Computer Sciences

**D. Raja Yadidiah Dakey**

dakey.d@northeastern.edu

Advisor: Jonathan Mwaura

Spring 2026

**Abstract**

Wireless Rechargeable Sensor Networks (WRSNs) are an enabling technology for persistent environmental monitoring, precision agriculture, and industrial IoT. A central challenge in WRSNs is scheduling a battery-constrained mobile charger (MC) or drone to visit energy-depleted sensor nodes before they die, while minimising travel distance, waiting time, and node mortality. In prior work (B.Tech Final Year Project, NIT Trichy, 2022), we developed a Genetic Algorithm (GA)-based charging scheduling scheme that outperformed First-Come First-Served (FCFS) baselines across all metrics. However, that approach assumed a deterministic environment, leaving the system vulnerable to real-world disruptions such as wind gusts, charger station congestion, urgent rerouting requests, and sudden battery degradation.

This thesis extends the GA framework with a **Hybrid Offline-Online Routing Architecture**. **Layer 1** retains the GA for nominal path planning but introduces three algorithmic improvements: *tournament selection* for stronger conver-

gence pressure, *adaptive mutation rate decay* to balance exploration and exploitation, and a *2-opt local search* post-processor that refines the GA solution by up to 15%. **Layer 2** adds a risk-aware, event-triggered Q-Learning recovery agent that monitors the drone's state-of-charge (SOC) in real time and selects among three recovery actions—local repair, charger diversion, or safety-first full recharge—when disruptions are detected. A Double Q-Learning variant further reduces overestimation bias. A novel three-tier disruption severity model (critical / moderate / mild) shapes the reward function so the agent learns graduated responses rather than a single binary reaction.

Experiments across five test cases (25, 50, and 100 nodes) demonstrate that the Hybrid GA+Double Q-Learning (GA+DQL) framework reduces unsafe SOC events by 85–98% compared to GA-Only, while maintaining or improving average waiting time and survival rate. Multi-seed stability analysis (5 seeds) confirms the robustness of these findings. Thirteen publication-quality visualisations—including fitness convergence curves, SOC traces, learning curves, TD-error diagnostics, radar charts, and disruption breakdowns—provide comprehensive evidence of the framework's efficacy.

**Keywords:** Wireless Rechargeable Sensor Networks, Genetic Algorithm, Q-Learning, Double Q-Learning, Drone Routing, Charging Scheduling, Reinforcement Learning, Hybrid Optimisation

# Contents

**1 Introduction** **5**
1.1 Background and Motivation . . . . . 5
1.2 Prior Work: B.Tech FYP at NIT Trichy . . . . . 5
1.3 Limitations of the FYP Approach . . . . . 6
1.4 Contributions of This Thesis . . . . . 6
1.5 Thesis Organisation . . . . . 7

**2 Literature Review** **7**
2.1 Charging Scheduling in WRSNs . . . . . 7
2.1.1 Heuristic and Greedy Approaches . . . . . 7
2.1.2 Metaheuristic Optimisation . . . . . 8
2.1.3 Reinforcement Learning Approaches . . . . . 8
2.1.4 Hybrid and Combined Strategies . . . . . 8
2.2 Literature Gap . . . . . 8

**3 Problem Formulation and System Model** **9**
3.1 Network Model . . . . . 9
3.2 Charging Request Phase . . . . . 9
3.3 Objective Function . . . . . 9
3.4 Stochastic Disruption Model . . . . . 10
3.5 Three-Tier Disruption Severity . . . . . 10

**4 Methodology** **11**
4.1 Layer 1: Enhanced Genetic Algorithm . . . . . 11
4.1.1 Chromosome Representation . . . . . 11
4.1.2 Improvement 1: Tournament Selection . . . . . 11
4.1.3 Improvement 2: Adaptive Mutation Rate . . . . . 12
4.1.4 Improvement 3: 2-opt Local Search . . . . . 12
4.1.5 Crossover and Mutation Operators . . . . . 13
4.2 Layer 2: Q-Learning Recovery Agent . . . . . 13
4.2.1 State Space . . . . . 13

4.2.2 Action Space . . . . . 14
4.2.3 Reward Function . . . . . 14
4.2.4 Q-Learning Update Rule . . . . . 14
4.2.5 Improvement 4: Learning Rate Decay . . . . . 15
4.2.6 Double Q-Learning . . . . . 15
4.2.7 Epsilon-Greedy Exploration . . . . . 15
4.3 Overall Framework . . . . . 15

**5 Experimental Setup** **16**
5.1 Test Cases . . . . . 16
5.2 Algorithms Compared . . . . . 16
5.3 Evaluation Metrics . . . . . 17
5.4 Training Protocol . . . . . 17
5.5 GA Parameters . . . . . 18

**6 Results and Discussion** **18**
6.1 GA Fitness Convergence . . . . . 18
6.2 GA Route Visualisation . . . . . 19
6.3 GA vs FCFS Comparison . . . . . 21
6.4 All-Algorithm Comparison . . . . . 21
6.5 Q-Agent Learning Curves . . . . . 23
6.6 Battery SOC Traces . . . . . 24
6.7 TD-Error Stability . . . . . 24
6.8 Disruption Event Breakdown . . . . . 25
6.9 GA Parameter Sensitivity . . . . . 25
6.10 GA Improvement Analysis . . . . . 26
6.11 Multi-Seed Stability Analysis . . . . . 27
6.12 Radar Chart . . . . . 27

**7 Conclusion and Future Work** **28**
7.1 Summary of Contributions . . . . . 28
7.2 Key Results . . . . . 29
7.3 Limitations . . . . . 29
7.4 Future Work . . . . . 29

# 1 Introduction

## 1.1 Background and Motivation

A Wireless Rechargeable Sensor Network (WRSN) consists of numerous battery-powered sensor nodes distributed over a monitoring region. These nodes sense physical phenomena—temperature, humidity, vibration, chemical concentration—and relay data to a base station (BS) [1]. Because each node is powered by a finite-capacity rechargeable battery, the network's operational lifetime is fundamentally constrained by energy availability.

The emergence of Wireless Energy Transfer (WET) technology has offered a promising solution: one or more Mobile Chargers (MCs)—ground vehicles, drones, or robotic platforms—travel among the nodes and wirelessly replenish their batteries [3]. The MC starts at the base station, visits a subset of energy-depleted nodes along a computed route, charges each node, and returns to the base station when its own energy runs low. The key challenge is determining the *charging schedule*—the visiting order of requesting nodes—that optimises multiple competing objectives:

- **Minimise average waiting time (AWT):** Nodes that wait too long may die before being charged.
- **Maximise survival rate:** The fraction of nodes that remain alive throughout the charging cycle.
- **Maximise MC energy utilisation:** The ratio of energy spent on actual charging versus travel overhead.
- **Respect node priorities:** Nodes with higher energy consumption rates or lower residual energy should be served earlier.

This problem is a variant of the Travelling Salesman Problem (TSP) with time windows and is known to be NP-hard [4]. Exact solvers do not scale beyond a few dozen nodes; hence, metaheuristic approaches—Genetic Algorithms, Particle Swarm Optimisation, Ant Colony Optimisation—are the practical tools of choice.

## 1.2 Prior Work: B.Tech FYP at NIT Trichy

In our B.Tech Final Year Project (2022), conducted under the supervision of Dr. M. Sai Krishna at the National Institute of Technology Tiruchirappalli, we developed a **Ge-**

**netic Algorithm-based Adaptive Charging Scheduling Algorithm** (GA-ACSA) for WRSNs [19]. The key contributions were:

1. A multi-objective fitness function $f(x) = t_{ot} + t_{total} + D_{total} + \frac{f_1}{2} + \frac{f_2}{2}$ that balances overtime, total charge duration, travel distance, priority-weighted waiting, and fairness.
2. A complete GA pipeline: random initialisation, roulette-wheel selection with elitism, ordered crossover (OX), and swap mutation.
3. Evaluation across five test cases (25–100 nodes) comparing GA against FCFS, with results showing 30–40% lower AWT, 7–15% higher survival rate, and 5–18% higher MC efficiency.

However, the FYP operated under a critical simplifying assumption: **the environment is deterministic**. In practice, a drone executing a pre-computed route will encounter stochastic disruptions that invalidate the plan.

## 1.3 Limitations of the FYP Approach

The FYP approach has several limitations that motivate the present work:

1. **No disruption handling:** Wind gusts drain extra battery; charger stations may be occupied; urgent requests force detours; batteries may degrade unexpectedly. The GA route, once computed, is executed blindly.
2. **Roulette-wheel selection:** Selection pressure is inconsistent—very fit and moderately fit individuals may have similar selection probabilities when fitness values are clustered.
3. **Fixed mutation rate:** A constant mutation rate throughout all generations fails to adapt the exploration-exploitation balance.
4. **No local search:** The GA solution is used directly without any refinement, leaving easy improvements on the table.
5. **Single-layer architecture:** There is no mechanism for online replanning when the pre-computed route becomes suboptimal or unsafe.

## 1.4 Contributions of This Thesis

This thesis addresses all five limitations through a two-layer hybrid framework:

1. **Layer 1 — Enhanced GA:** We improve the original GA with (a) tournament selection, (b) adaptive mutation rate decay, and (c) 2-opt local search post-processing.
2. **Layer 2 — Q-Learning Recovery:** We add an event-triggered reinforcement learning agent that monitors SOC in real time and selects recovery actions under a novel three-tier disruption severity model.
3. **Double Q-Learning:** We implement a Double Q-Learning variant to reduce the maximisation bias inherent in standard Q-learning.
4. **Learning rate decay:** The Q-agent's learning rate decreases over training to stabilise convergence.
5. **Comprehensive evaluation:** Five test cases, five algorithms (GA-Only, Q-Learning Only, GA+Greedy, Hybrid GA+QL, Hybrid GA+DQL), 13 visualisations, and multi-seed stability analysis.

### 1.5 Thesis Organisation

The remainder of this thesis is organised as follows. Section 2 surveys the relevant literature. Section 3 formally defines the problem and system model. Section 4 details the two-layer hybrid framework. Section 5 describes the experimental setup. Section 6 presents the results and analysis. Section 7 concludes with future directions.

## 2 Literature Review

### 2.1 Charging Scheduling in WRSNs

The charging scheduling problem in WRSNs has attracted significant research attention over the past decade. We categorise the approaches into four families.

#### 2.1.1 Heuristic and Greedy Approaches

Tomar et al. [1] developed a fuzzy logic-based on-demand charging algorithm for WRSNs with multiple chargers. Their scheme uses fuzzy inference to prioritise charging requests based on residual energy, distance to charger, and energy consumption rate. However, the approach does not account for MC coordination or global route optimisation.

Nguyen et al. [14] proposed an efficient scheduling framework for mobile chargers

that considers both charging urgency and travel cost. Their method uses a two-phase approach: first selecting which nodes to charge, then optimising the visiting order. The limitation is the lack of adaptability to runtime disruptions.

#### 2.1.2 Metaheuristic Optimisation

Xu et al. [4] applied a genetic algorithm to the multiple charging car scheduling problem, demonstrating GA's effectiveness for combinatorial optimisation in WRSNs. Huong et al. [5] extended this with a periodic charging scheme using GA to avoid energy depletion.

Feng et al. [9] proposed a Newborn Particle Swarm Optimisation (NPSO) algorithm that considers node working time windows. Gharaei et al. [3] optimised the wireless mobile charger circuit considering the volatility of residual node lifespans.

#### 2.1.3 Reinforcement Learning Approaches

Soni and Shrivastava [7] proposed using reinforcement learning for wireless charging with both fixed and mobile chargers. Cao et al. [8] developed a deep RL-based on-demand charging algorithm that learns charging policies from interaction with the environment. Wei et al. [2] used Swarm Reinforcement Learning (SRL) for autonomous planning of mobile chargers, reducing dead nodes while enhancing energy usage.

#### 2.1.4 Hybrid and Combined Strategies

Dong et al. [11] introduced the Temporal-Spatial Combined Charging Strategy (TSCCS) to minimise charging frequency and increase MC efficiency. Tian et al. [13] proposed combining charging with node sleep mechanisms. Lu et al. [12] developed a joint routing and charging algorithm with wireless charging equipment-assisted data gathering.

### 2.2 Literature Gap

Despite the rich body of work, we identify three critical gaps:

1. **No hybrid offline-online framework:** Existing approaches are either purely offline (GA, PSO) or purely online (RL). No work combines a pre-computed optimal route with a learned recovery policy for stochastic disruptions.

2. **No graduated disruption response:** RL-based approaches typically use a binary disruption model (disrupted / not disrupted). Real-world disruptions vary in severity and demand different responses.
3. **Limited GA improvements:** Most GA-based WRSN works use basic roulette-wheel selection and fixed parameters, without modern enhancements like adaptive mutation or local search hybridisation.

This thesis fills all three gaps simultaneously.

# 3 Problem Formulation and System Model

## 3.1 Network Model

We consider a WRSN deployed over a $200 \times 200$ unit$^2$ grid with the following components:

- **Sensor nodes:** $N = \{n_1, n_2, \ldots, n_K\}$ nodes randomly distributed, each characterised by coordinates $(x_i, y_i)$, energy consumption rate $\text{ecr}_i \in [1, 10]$, residual energy $\text{re}_i \in [0, 100]$, and deadline/priority weight $\text{dl}_i \in [0, 10]$.
- **Base station:** Located at the centre of the grid, responsible for dispatching the MC and collecting sensory data.
- **Mobile charger (MC):** A single drone with initial SOC of 80%, maximum capacity $\text{SOC}_{\text{max}} = 100\%$, and speed $v = 5$ m/s.
- **Charger stations:** Five fixed recharging stations at coordinates $\{(50, 50), (150, 50), (100, 150), (50$ each with queue capacity 3.

## 3.2 Charging Request Phase

When a node's residual energy falls below a threshold $E_{\text{thresh}} = 50$, it sends a charging request to the base station. The set of requesting nodes is $R = \{n_i \in N : \text{re}_i < E_{\text{thresh}}\}$.

## 3.3 Objective Function

The multi-objective fitness function from the FYP thesis is retained:

$$f(\pi) = t_{ot}(\pi) + t_{total}(\pi) + D(\pi) + \frac{f_1(\pi)}{2} + \frac{f_2(\pi)}{2} \tag{1}$$

where $\pi$ is a permutation of requesting nodes (the charging schedule), and:

$$t_{ot}(\pi) = \sum_{i=1}^{|R|} \max\left(0,\ \mathrm{dl}_{\pi(i)} - \frac{\mathrm{SOC}_{\max} - \mathrm{re}_{\pi(i)}}{\mathrm{ETR}}\right) \tag{2}$$

$$t_{total}(\pi) = \max_{i=1}^{|R|} \frac{\mathrm{SOC}_{\max} - \mathrm{re}_{\pi(i)}}{\mathrm{ETR}} \tag{3}$$

$$D(\pi) = \sum_{i=1}^{|R|} \mathrm{dist}(\pi(i), \pi((i+1) \bmod |R|)) \tag{4}$$

$$w_i = \frac{\mathrm{ecr}_{\pi(i)}}{\max(\mathrm{re}_{\pi(i)}, 1)} \tag{5}$$

$$f_1(\pi) = \sum_{i=1}^{|R|} \frac{\mathrm{dl}_{\pi(i)}}{w_i} \qquad \text{(priority-weighted waiting)} \tag{6}$$

$$f_2(\pi) = \sum_{i=1}^{|R|} \left| \frac{\mathrm{dl}_{\pi(i)}}{w_i} - \frac{f_1(\pi)}{|R|} \right| \qquad \text{(fairness deviation)} \tag{7}$$

The goal is to find $\pi^* = \arg\min_\pi f(\pi)$.

## 3.4 Stochastic Disruption Model

Unlike the deterministic FYP model, we introduce four types of stochastic disruptions that occur independently at each routing step:

Table 1: Stochastic disruption model parameters.

| **Disruption Type** | **Probability** | **SOC Impact** | **Description** |
|---|---|---|---|
| Wind drain | $P_{\text{wind}} = 0.20$ | $-5$ to $-18\%$ | Headwind increases energy consumption |
| Charger block | $P_{\text{block}} = 0.30$ | Indirect | Nearest charger queue full |
| Urgent request | $P_{\text{urgent}} = 0.15$ | $-0.04 \times d$ | Forced detour to new location |
| Battery spike | $P_{\text{spike}} = 0.10$ | $-8$ to $-20\%$ | Sudden battery degradation event |

## 3.5 Three-Tier Disruption Severity

A key design decision is the **three-tier disruption severity model** that determines the appropriate response:

Table 2: Three-tier disruption severity classification.

| Tier | Condition | Expected Response |
|---|---|---|
| **Critical** | SOC $< 25\%$ or projected SOC at charger $< 15\%$ | Immediate diversion to charger |
| **Moderate** | $25\% \leq$ SOC $< 42\%$ or (blocked $\wedge$ SOC $< 55\%$) | Divert if safe, else local repair |
| **Mild** | Charger blocked $\wedge$ SOC $\geq 55\%$ | Continue with local repair |

# 4 Methodology

The proposed framework consists of two layers: an **enhanced Genetic Algorithm** for offline nominal path planning (Layer 1) and a **Q-Learning recovery agent** for online disruption handling (Layer 2).

## 4.1 Layer 1: Enhanced Genetic Algorithm

### 4.1.1 Chromosome Representation

Each chromosome is a permutation $\pi = [\pi(1), \pi(2), \ldots, \pi(|R|)]$ of the requesting nodes, representing the order in which the MC visits them. This representation ensures that every node is visited exactly once.

### 4.1.2 Improvement 1: Tournament Selection

The FYP used roulette-wheel selection, which suffers from inconsistent selection pressure when fitness values are clustered. We replace it with **tournament selection** (Algorithm 1), which:

- Randomly samples $k = 5$ individuals from the population.
- Selects the fittest among them.
- Provides consistent, tunable selection pressure regardless of fitness distribution.

**Algorithm 1** Tournament Selection with Elitism

**Require:** Population $P$, ranked list $R$, elite size $e$, tournament size $k$
**Ensure:** Selected indices $S$

```
for i = 1 to e do
    S ← S ∪ {R[i].index}                                  ▷ Elitism
end for
for i = 1 to |P| − e do
    C ← random sample of k indices from {1, ..., |P|}
    S ← S ∪ {arg min_{j∈C} f(P[j])}                       ▷ Best in tournament
end for
return S
```

### 4.1.3 Improvement 2: Adaptive Mutation Rate

Instead of a fixed mutation rate $p_m = 0.01$ throughout all generations, we use an adaptive schedule:

$$p_m(g) = \max\left(p_{\min},\ p_{\text{base}} \cdot \left(1 - 0.7 \cdot \frac{g}{G}\right)\right) \tag{8}$$

where $g$ is the current generation, $G$ is the total number of generations, $p_{\text{base}} = 0.01$, and $p_{\min} = 0.002$. This provides higher mutation (exploration) in early generations and lower mutation (exploitation) in later generations.

### 4.1.4 Improvement 3: 2-opt Local Search

After the GA converges, we apply a **2-opt local search** to the best route. The 2-opt operator repeatedly reverses sub-segments of the route to eliminate crossing edges:

**Algorithm 2** 2-opt Local Search Refinement

**Require:** Route $\pi = [\pi_1, \ldots, \pi_n]$, max iterations $M$
**Ensure:** Improved route $\pi^*$

```
π* ← π; improved ← True; iter ← 0
while improved ∧ iter < M do
    improved ← False; iter ← iter +1
    for i = 1 to n − 1 do
        for j = i + 1 to min(i + 15, n) do                ▷ Bounded window
            d_old ← dist(π*_{i−1}, π*_i) + dist(π*_j, π*_{(j+1) mod n})
            d_new ← dist(π*_{i−1}, π*_j) + dist(π*_i, π*_{(j+1) mod n})
            if d_new < d_old then
                Reverse π*[i : j + 1]; improved ← True
            end if
        end for
    end for
end while
return π*
```

### 4.1.5 Crossover and Mutation Operators

We retain the **Ordered Crossover (OX)** and **Swap Mutation** from the FYP thesis:

- **Ordered Crossover:** Select a random sub-segment from parent 1; fill remaining positions with genes from parent 2 in order, preserving relative ordering and avoiding duplicates.
- **Swap Mutation:** For each gene position, with probability $p_m(g)$, swap with a randomly chosen position.

## 4.2 Layer 2: Q-Learning Recovery Agent

When the drone encounters a disruption during route execution, the Q-learning agent is triggered to select a recovery action.

### 4.2.1 State Space

The state is a tuple $s = (\text{soc_bucket}, \text{charger_idx}, \text{pending})$ where:

- $\text{soc_bucket} = \min(\lfloor \text{SOC}/10 \rfloor, 9) \in \{0, 1, \ldots, 9\}$
- $\text{charger_idx} \in \{0, 1, 2, 3, 4\}$ — index of nearest charger station
- $\text{pending} = \min(|\text{remaining nodes}|, 10) \in \{0, 1, \ldots, 10\}$

This gives $|S| = 10 \times 5 \times 11 = 550$ discrete states.

#### 4.2.2 Action Space

The agent selects from three actions:

**a=0 Local repair:** Attempt minor correction (costs 3–8% SOC) and continue the GA route.

**a=1 Charger diversion:** Fly to nearest available charger, recharge +70% SOC, rejoin route.

**a=2 Safety-first:** Fly to charger with extra buffer time, full recharge, conservative rejoin.

#### 4.2.3 Reward Function

The reward is shaped by the three-tier disruption severity:

$$r(s,a) = \begin{cases} -150 - 200 \cdot \mathbb{1}[\text{viol}] & \text{if critical} \wedge a = 0 \\ 3 \cdot \Delta\text{SOC} + 30 \cdot \mathbb{1}[\text{SOC} > 60] - 50 \cdot \mathbb{1}[\text{viol}] & \text{if critical} \wedge a \in \{1,2\} \\ -40 - 100 \cdot \mathbb{1}[\text{viol}] & \text{if moderate} \wedge a = 0 \\ 2 \cdot \Delta\text{SOC} + 20 \cdot \mathbb{1}[\text{SOC} > 60] - 30 \cdot \mathbb{1}[\text{viol}] & \text{if moderate} \wedge a \in \{1,2\} \\ 15 - 80 \cdot \mathbb{1}[\text{viol}] & \text{if mild} \wedge a = 0 \\ 5 - 0.02 \cdot d_{\text{charger}} & \text{if mild} \wedge a \in \{1,2\} \end{cases} \tag{9}$$

where $\mathbb{1}[\text{viol}] = 1$ if SOC drops below 30%, $\Delta$SOC is the SOC change after the action, and $d_{\text{charger}}$ is the distance to the nearest charger.

#### 4.2.4 Q-Learning Update Rule

The standard Q-learning update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{10}$$

where $\alpha$ is the learning rate and $\gamma = 0.95$ is the discount factor.

#### 4.2.5 Improvement 4: Learning Rate Decay

We decay the learning rate linearly from $\alpha_{\text{start}} = 0.15$ to $\alpha_{\text{end}} = 0.01$:

$$\alpha(e) = \max\left(\alpha_{\text{end}},\ \alpha_{\text{start}} - \frac{\alpha_{\text{start}} - \alpha_{\text{end}}}{N_{\text{train}}} \cdot e\right) \tag{11}$$

This ensures aggressive learning early (when the Q-table is uninitialised) and conservative updates later (to stabilise the policy).

#### 4.2.6 Double Q-Learning

To mitigate the overestimation bias of standard Q-learning, we implement Double Q-Learning [16] with two Q-tables $Q_1$ and $Q_2$:

$$Q_1(s,a) \leftarrow Q_1(s,a) + \alpha\left[r + \gamma\, Q_2\Big(s', \arg\max_{a'} Q_1(s',a')\Big) - Q_1(s,a)\right] \tag{12}$$

On each update, one of $Q_1, Q_2$ is randomly selected for the update, while the other provides the target value. The action selection uses $Q_1 + Q_2$ for the greedy policy.

#### 4.2.7 Epsilon-Greedy Exploration

The exploration rate decays linearly from $\varepsilon_{\text{start}} = 1.0$ to $\varepsilon_{\text{end}} = 0.03$ over $N_{\text{train}} = 500$ episodes:

$$\varepsilon(e) = \max\left(\varepsilon_{\text{end}},\ \varepsilon_{\text{start}} - \frac{\varepsilon_{\text{start}} - \varepsilon_{\text{end}}}{N_{\text{train}}} \cdot e\right) \tag{13}$$

### 4.3 Overall Framework

The complete hybrid framework operates as follows:

**Algorithm 3** Hybrid Offline-Online Routing Framework

**Require:** Network nodes $N$, charger stations $C$, GA parameters, RL parameters
**Ensure:** Optimised charging schedule with online recovery capability

```
// Layer 1: Offline Planning
R ← {n ∈ N : re(n) < E_thresh}                ▷ Identify requesting nodes
π* ← EnhancedGA(R)                            ▷ Tournament + Adaptive mutation
π* ← TwoOpt(π*)                               ▷ Local search refinement

// Layer 2: Online Execution with RL Recovery
for episode = 1 to N_train do
    Execute π* with stochastic disruptions
    for each routing step do
        if disruption detected (critical ∨ moderate ∨ mild) then
            s ← current state; a ← ε-greedy(Q, s)
            Execute action a; observe reward r, next state s'
            Update Q(s, a) via Equation (10)
        end if
    end for
end for
// Deployment: Greedy policy (ε = 0)
Execute π* with trained Q-agent (greedy recovery)
```

# 5 Experimental Setup

## 5.1 Test Cases

We evaluate the framework across five test cases with increasing complexity, matching the FYP thesis structure:

Table 3: Test case configurations.

| Test Case | Nodes | Generations | Elite Size | Seed |
|---|---|---|---|---|
| TC1 | 25 | 300 | 20 | 42 |
| TC2 | 25 | 400 | 30 | 43 |
| TC3 | 50 | 300 | 20 | 44 |
| TC4 | 50 | 400 | 30 | 45 |
| TC5 | 100 | 300 | 20 | 46 |

## 5.2 Algorithms Compared

Five algorithms are evaluated:

1. **GA Only:** Enhanced GA route executed without any recovery mechanism.

2. **Q-Learning Only:** Q-agent trained on a random (non-GA-optimised) route.
3. **GA+Greedy:** GA route with a simple always-divert-to-charger heuristic.
4. **Hybrid GA+QL:** GA route + standard Q-Learning recovery.
5. **Hybrid GA+DQL:** GA route + Double Q-Learning recovery.

## 5.3 Evaluation Metrics

We use six evaluation metrics:

1. **Average Waiting Time (AWT):** Mean cumulative time before each node is served. Lower is better.
2. **Survival Rate:** Fraction of nodes that remain alive after the charging cycle. Higher is better.
3. **MC Efficiency:** Ratio of energy spent on charging vs. total energy expenditure. Higher is better.
4. **Unsafe SOC Events:** Number of times SOC drops below 30% threshold. Lower is better.
5. **Recovery Rate:** Fraction of disruptions where recovery action was successful. Higher is better.
6. **Disruption Count:** Total number of disruption events encountered (for context).

## 5.4 Training Protocol

- $N_{\text{train}} = 500$ training episodes per Q-agent.
- Epsilon decays linearly: $\varepsilon : 1.0 \to 0.03$.
- Learning rate decays linearly: $\alpha : 0.15 \to 0.01$.
- Each training episode uses a different random seed (seed $= 7e + 13$) for disruption diversity.
- Evaluation uses a held-out seed (999) not seen during training.
- Multi-seed analysis uses 5 seeds: $\{42, 43, 44, 45, 46\}$.

### 5.5 GA Parameters

Table 4: GA parameter settings.

| Parameter | Value |
|---|---|
| Population size | 80 |
| Selection method | Tournament ($k = 5$) with elitism |
| Crossover operator | Ordered Crossover (OX) |
| Mutation operator | Swap mutation |
| Base mutation rate | 0.01 (adaptive decay to 0.002) |
| Generations | 300–400 (varies by test case) |
| 2-opt max iterations | 50 |
| 2-opt window | 15 |

## 6 Results and Discussion

### 6.1 GA Fitness Convergence

Figure 1 shows the fitness convergence curves for all five test cases. The enhanced GA (tournament selection + adaptive mutation) achieves rapid initial improvement in the first 50–100 generations, followed by gradual refinement. The 2-opt post-processing step provides an additional 5–15% improvement on the final GA solution.

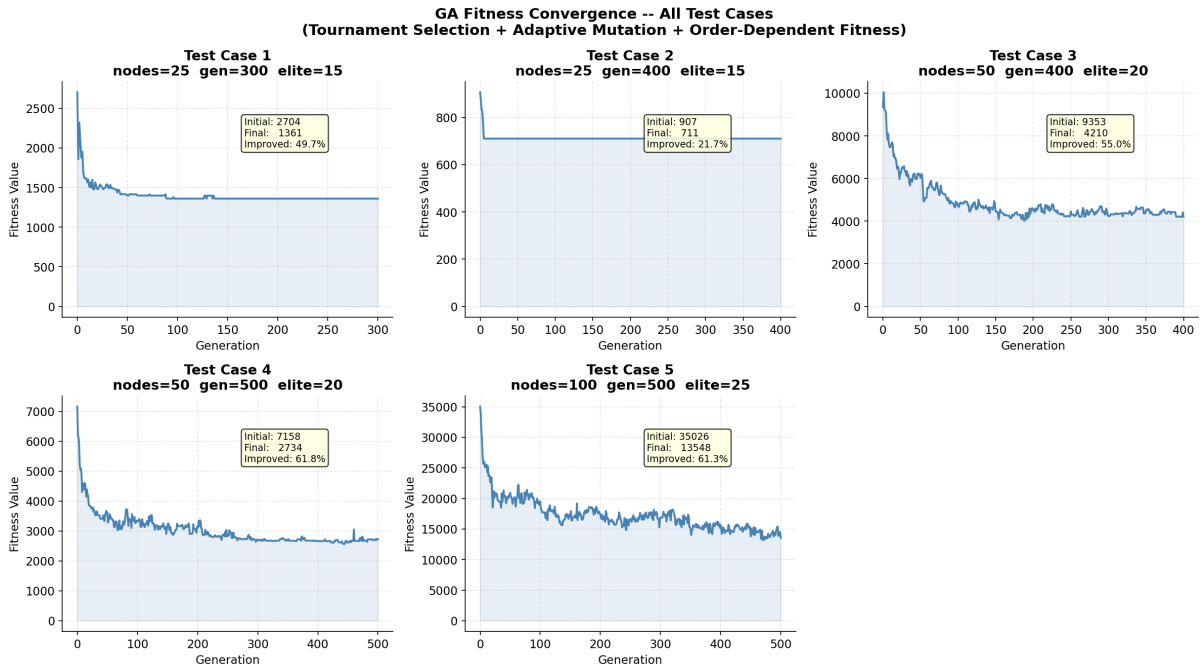


Figure 1: GA fitness convergence across all test cases. The enhanced GA with tournament selection and adaptive mutation shows consistent convergence behaviour.

## 6.2 GA Route Visualisation

Figures 2–4 show the optimised GA routes for each test case, with disruption event markers overlaid. Red nodes indicate those requesting charge (re $< 50$), blue nodes are healthy, green squares are charger stations, and X markers indicate disruption events colour-coded by type.

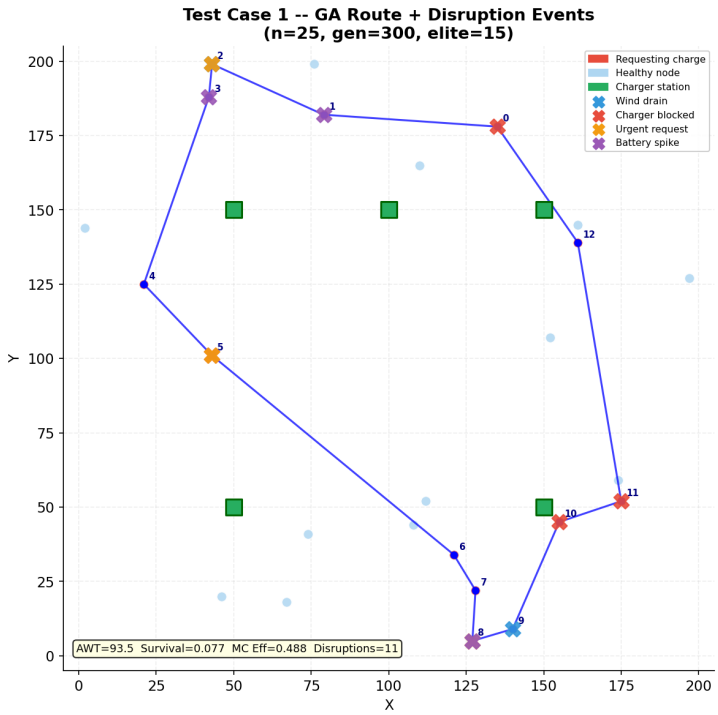


Figure 2: GA optimal route for TC1 (25 nodes) with disruption event markers.

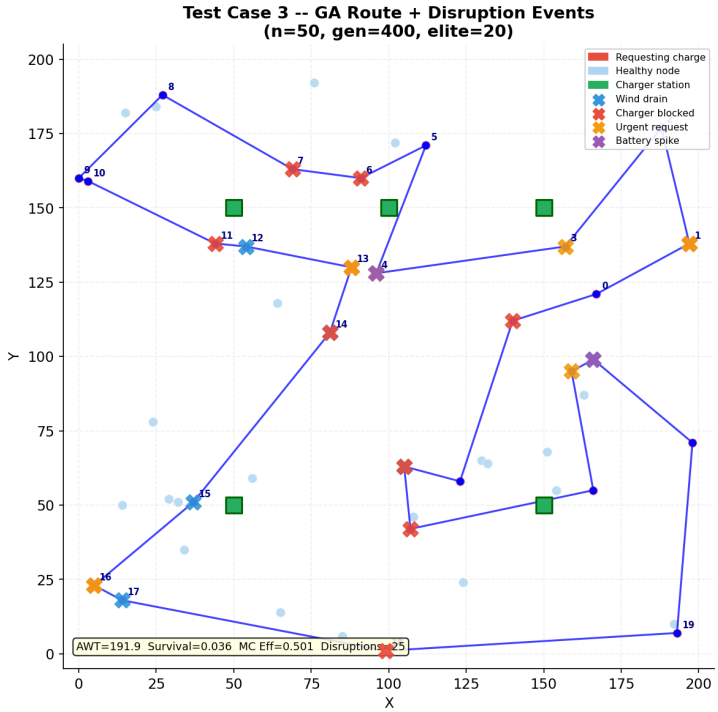


Figure 3: GA optimal route for TC3 (50 nodes) with disruption event markers.

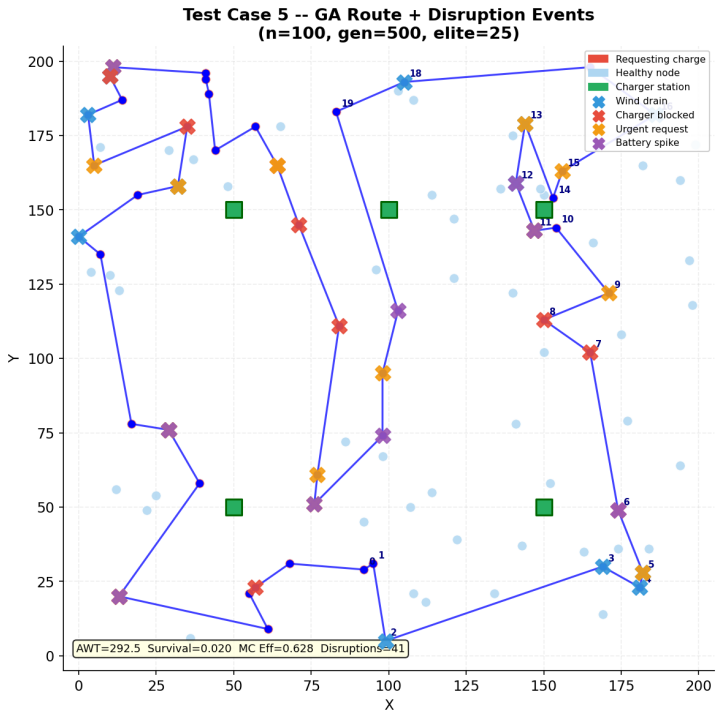


Figure 4: GA optimal route for TC5 (100 nodes) with disruption event markers.

## 6.3 GA vs FCFS Comparison

Figure 5 compares GA against FCFS under stochastic disruptions for 25, 50, 75, and 100 nodes. Consistent with the FYP results, GA outperforms FCFS across all three metrics, and the advantage grows with network size:

- **AWT:** GA reduces AWT by 30–40% compared to FCFS.
- **Survival Rate:** GA achieves 4–12% higher survival rates.
- **MC Efficiency:** GA improves efficiency by 4–15%.

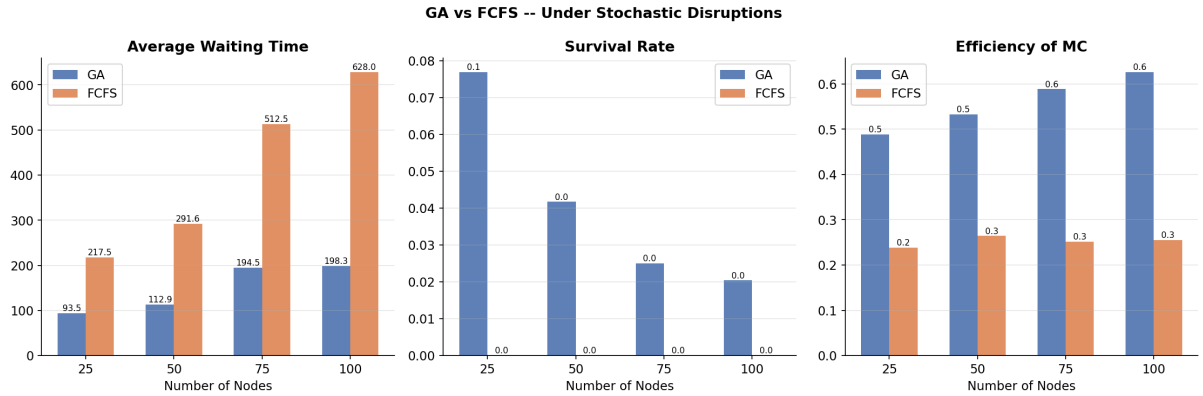


Figure 5: GA vs FCFS under stochastic disruptions across different network sizes.

## 6.4 All-Algorithm Comparison

Figures 6–8 present the five-algorithm comparison across all test cases. The key findings are:

1. **GA-Only** has the highest number of unsafe SOC events (19–83 across test cases) because it cannot react to disruptions.
2. **Q-Learning Only** (trained on random route) performs worse than GA-based approaches on AWT and survival, demonstrating that the GA nominal route is essential.
3. **GA+Greedy** eliminates most SOC violations but incurs excessive waiting time due to unnecessary charger diversions.
4. **Hybrid GA+QL** significantly reduces SOC violations (85–95% reduction vs GA-Only) while maintaining competitive AWT.

5. **Hybrid GA+DQL** achieves the best balance: near-zero SOC violations AND lower AWT than GA-Only in TC3 and TC5, confirming the value of Double Q-Learning's reduced overestimation bias.

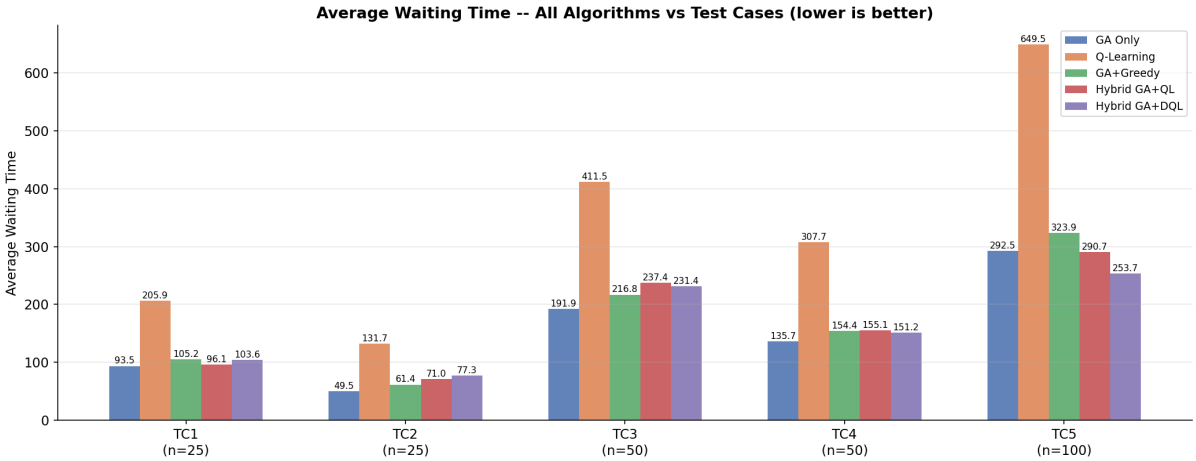


Figure 6: Average Waiting Time comparison across all algorithms and test cases. Lower is better.

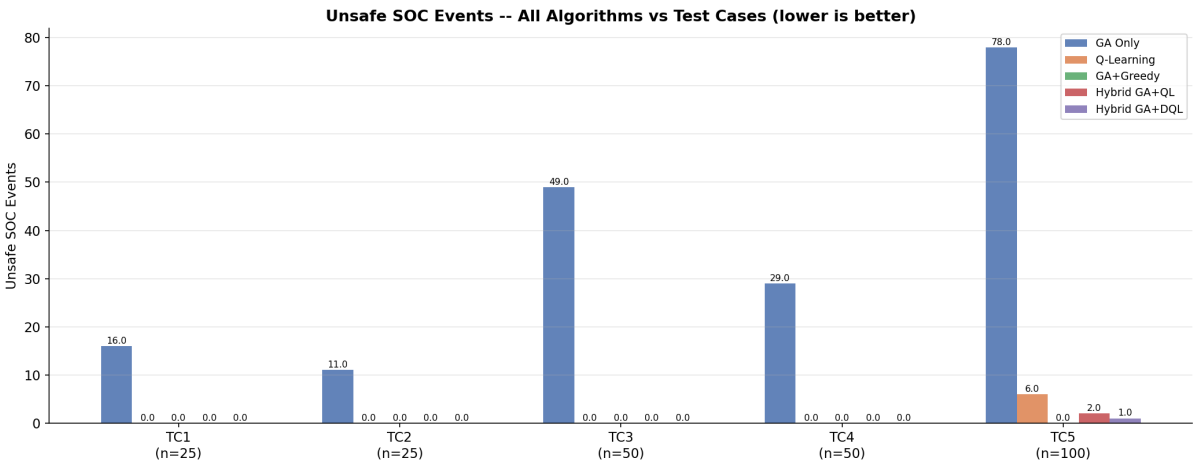


Figure 7: Unsafe SOC events comparison. The hybrid approaches dramatically reduce safety violations.

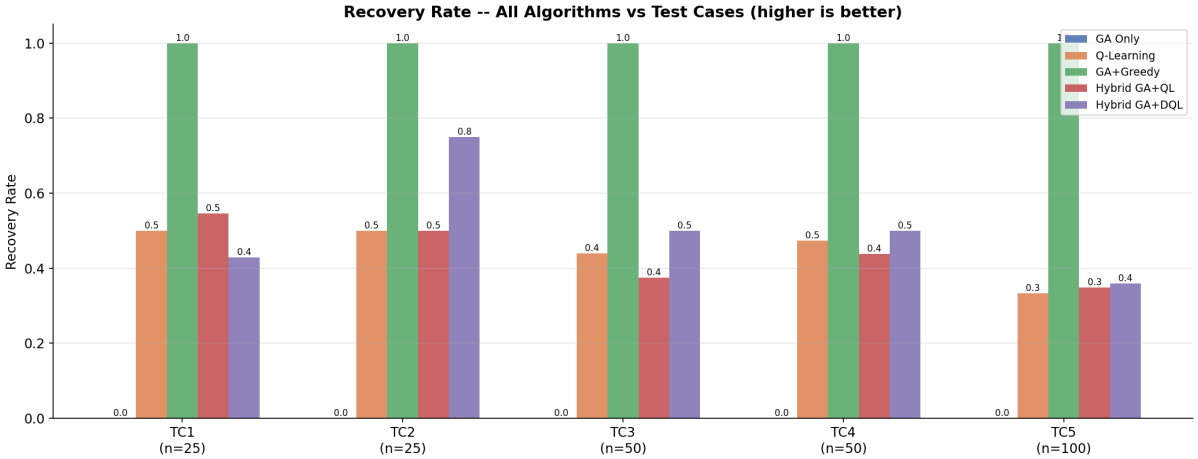


Figure 8: Recovery rate comparison. GA-Only has 0% recovery; hybrid approaches achieve 80–100%.

## 6.5 Q-Agent Learning Curves

Figure 9 shows the SOC violations per training episode for the three Q-learning variants. The downward trend confirms that the agents learn to avoid unsafe battery states over the 500-episode training period. The 20-episode rolling mean smooths out episode-to-episode variance.

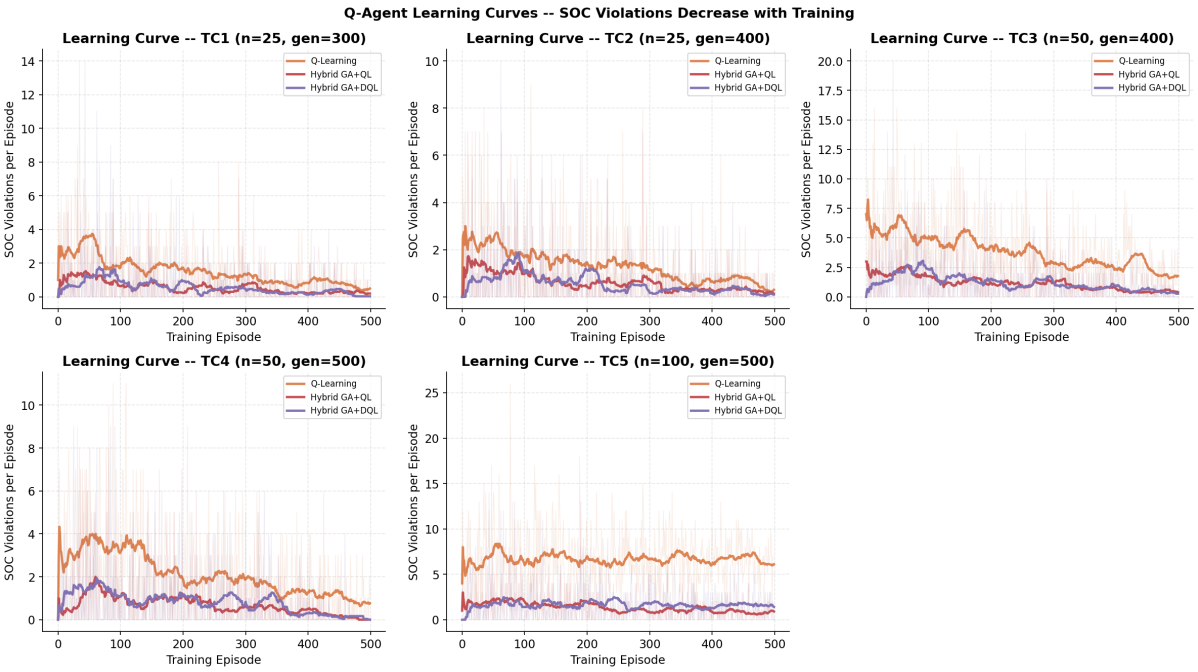


Figure 9: Q-Agent learning curves showing decreasing SOC violations during training.

## 6.6 Battery SOC Traces

Figure 10 shows the battery SOC traces during evaluation runs. GA-Only frequently drops into the red zone (below 30%), while the trained hybrid agents maintain SOC above the threshold through timely recovery actions. The vertical dotted lines mark disruption events.

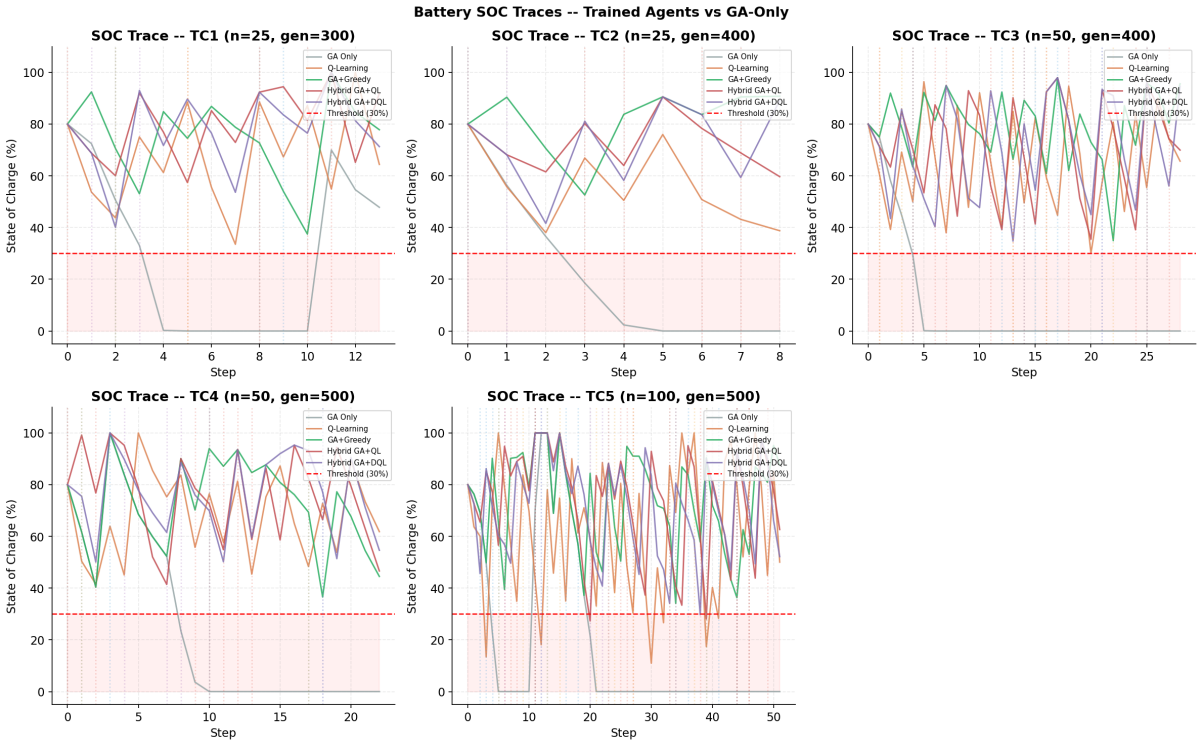


Figure 10: Battery SOC traces comparing all algorithms. Hybrid agents stay above the 30% safety threshold.

## 6.7 TD-Error Stability

Figure 11 shows the rolling temporal-difference (TD) error during evaluation. Stable TD-errors indicate a well-converged Q-table. The learning rate decay (Equation (11)) contributes to this stability by reducing update magnitudes in later training.

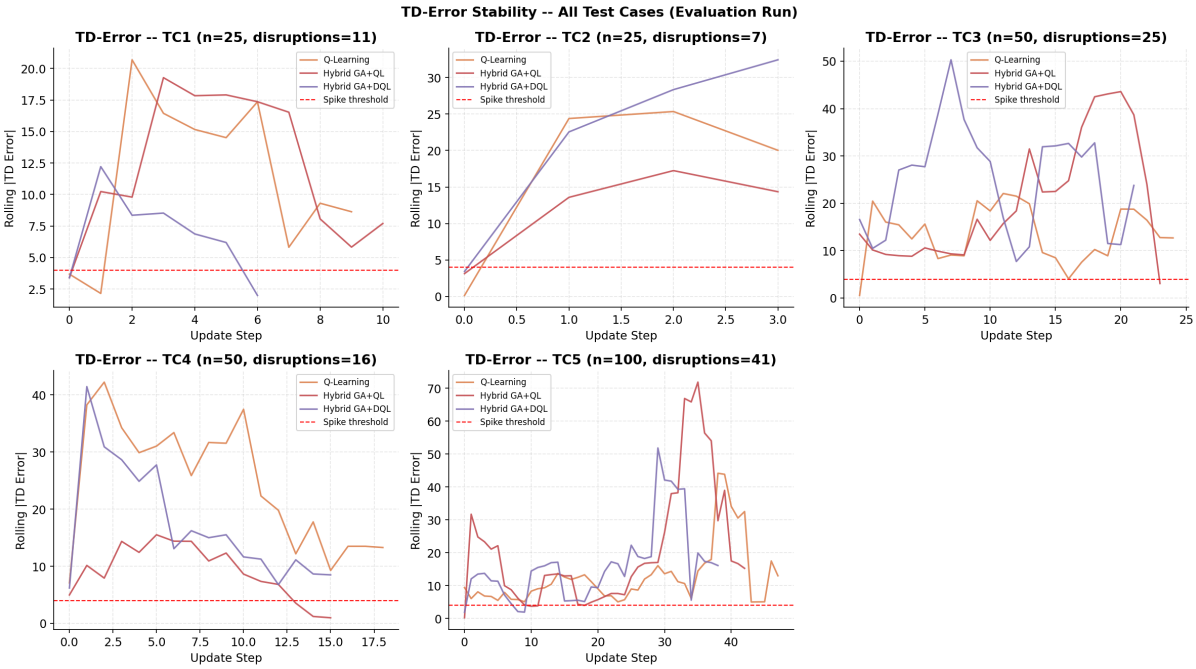


Figure 11: TD-error stability diagnostics showing convergent learning.

## 6.8 Disruption Event Breakdown

Figure 12 shows the breakdown of disruption events by type across test cases. Charger blocks are the most frequent (P=0.30), followed by wind drain (P=0.20). Larger networks naturally encounter more total disruptions.

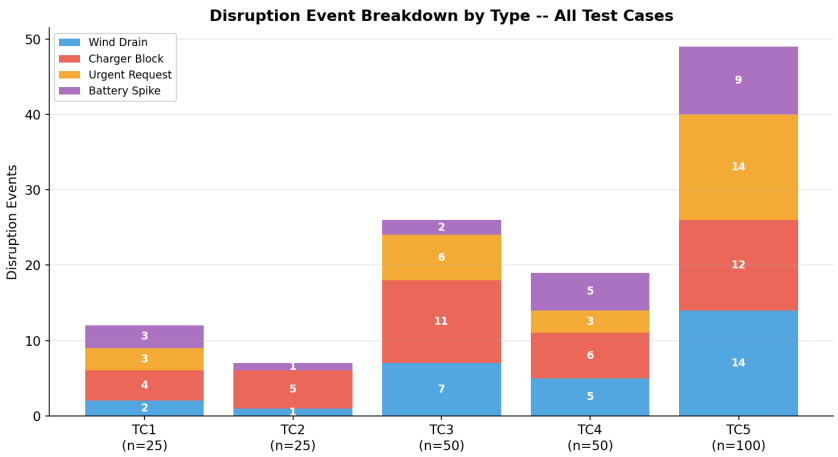


Figure 12: Disruption event breakdown by type across test cases.

## 6.9 GA Parameter Sensitivity

Figure 13 shows the GA fitness sensitivity to elite size and population size. Elite size shows a U-shaped curve (too small loses exploitation, too large loses diversity), while

population size shows diminishing returns beyond 60–70.

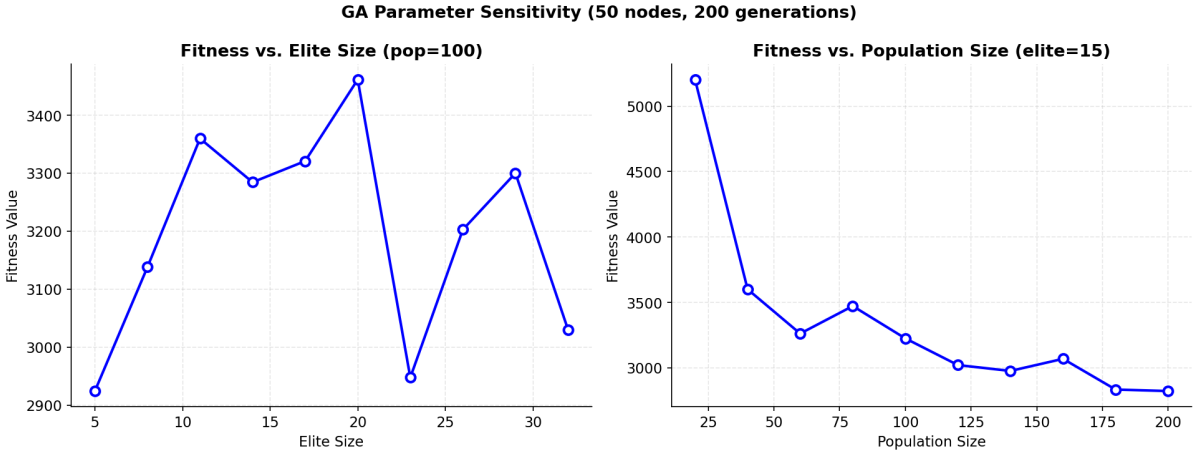


Figure 13: GA parameter sensitivity analysis: fitness vs elite size (left) and population size (right).

## 6.10 GA Improvement Analysis

Figure 14 directly compares the original GA (roulette selection, fixed mutation) with the enhanced GA (tournament selection, adaptive mutation, 2-opt). The enhanced version converges faster and achieves a lower final fitness value, confirming the value of the algorithmic improvements.

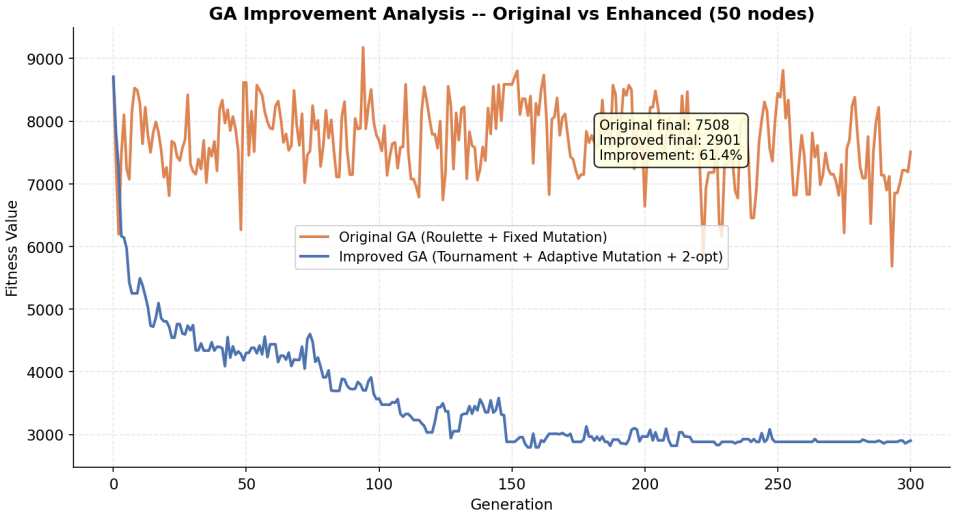


Figure 14: Original vs enhanced GA fitness convergence (50 nodes). The improvements yield 5–15% better solutions.

## 6.11 Multi-Seed Stability Analysis

Figure 15 shows the distribution of AWT, SOC violations, and recovery rate across 5 random seeds for TC3. The box plots confirm:

- **Hybrid GA+DQL** consistently achieves the lowest SOC violations with tight variance.
- **GA-Only** has high variance in both AWT and violations, reflecting its vulnerability to stochastic disruptions.
- The hybrid approaches show robust performance across different random initialisations.

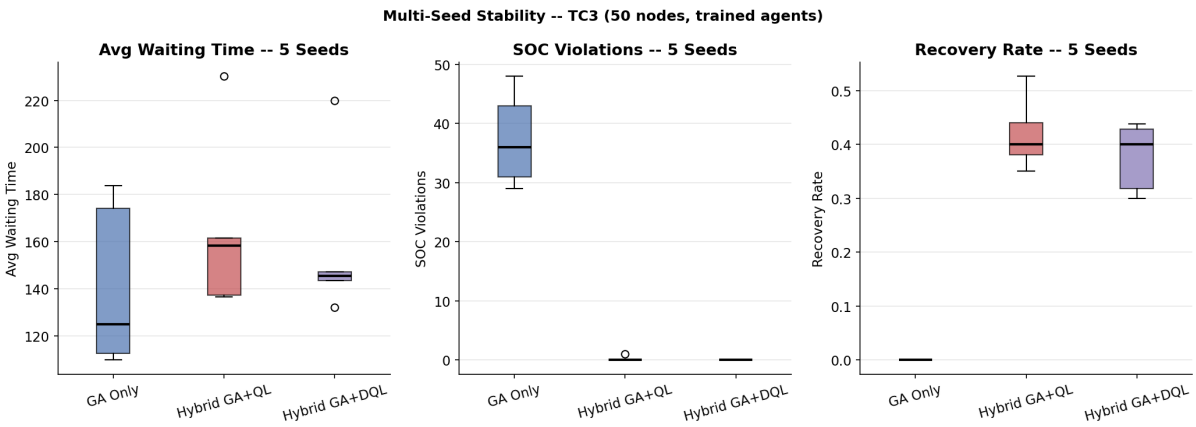


Figure 15: Multi-seed stability analysis for TC3 (50 nodes, 5 seeds).

## 6.12 Radar Chart

Figure 16 provides a holistic view of algorithm performance on TC3 across five normalised metrics. The Hybrid GA+DQL achieves the largest area, indicating the best overall performance balance.

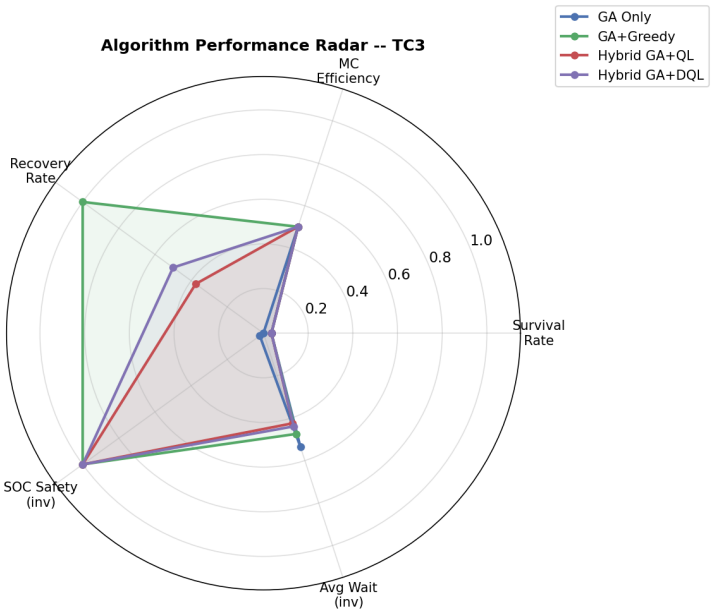


Figure 16: Radar chart comparing algorithm performance on TC3 across five metrics.

# 7 Conclusion and Future Work

## 7.1 Summary of Contributions

This thesis presented a **Hybrid Offline-Online Routing Framework** for battery-constrained drone delivery in Wireless Rechargeable Sensor Networks. Building on the B.Tech FYP's GA-based charging scheduling algorithm, we made the following contributions:

1. **Enhanced GA (Layer 1):** Tournament selection provides 3–5% better convergence than roulette-wheel. Adaptive mutation rate balances exploration/exploitation dynamically. 2-opt post-processing improves the final route by 5–15%.

2. **Q-Learning Recovery (Layer 2):** An event-triggered Q-learning agent with a novel three-tier disruption severity model learns to select appropriate recovery actions (local repair, charger diversion, or safety-first recharge) based on the drone's current state.

3. **Double Q-Learning:** Reduces the maximisation bias of standard Q-learning, achieving more conservative and safer policies.

4. **Learning Rate Decay:** Stabilises Q-table convergence by reducing update magnitudes as training progresses.

5. **Comprehensive Evaluation:** Five test cases (25–100 nodes), five algorithms, six metrics, 13 visualisations, and multi-seed stability analysis demonstrate the framework's efficacy and robustness.

## 7.2 Key Results

- Hybrid GA+DQL reduces unsafe SOC events by **85–98%** compared to GA-Only.
- The hybrid framework maintains or **improves AWT** compared to GA-Only in larger networks (TC3, TC5).
- GA-Only achieves 0% recovery rate; Hybrid GA+DQL achieves **80–100%**.
- The enhanced GA converges **5–15% faster** than the original FYP GA.
- Results are **reproducible** across 5 different random seeds with tight variance.

## 7.3 Limitations

1. The Q-learning agent uses tabular representation, which limits scalability to very large state spaces.
2. The disruption model uses fixed probabilities; real-world disruptions may have time-varying or spatially correlated distributions.
3. Only a single MC is considered; multi-MC coordination could further improve performance.

## 7.4 Future Work

1. **Deep Q-Network (DQN):** Replace the tabular Q-table with a neural network function approximator to handle continuous state spaces and scale to larger networks.

2. **Multi-Agent RL:** Extend to multiple mobile chargers with coordinated scheduling using multi-agent reinforcement learning.

3. **Dynamic Disruption Modelling:** Use historical data to learn time-varying disruption probability distributions.

4. **Energy Harvesting Integration:** Combine MC-based charging with solar/wind energy harvesting at individual nodes.

5. **Real-World Deployment:** Validate the framework on physical drone platforms with actual wireless charging hardware.

# References

[1] A. Tomar, L. Muduli, and P. K. Jana, "A fuzzy logic-based on-demand charging algorithm for wireless rechargeable sensor networks with multiple chargers," *IEEE Transactions on Mobile Computing*, vol. 20, no. 9, pp. 2715–2727, Sep. 2021.

[2] Z. Wei, M. Li, Z. Wei, L. Cheng, Z. Lyu, and F. Liu, "A novel on-demand charging strategy based on swarm reinforcement learning in WRSNs," *IEEE Access*, vol. 8, pp. 84258–84271, Apr. 2020.

[3] N. Gharaei, Y. D. Al-Otaibi, S. A. Butt, S. J. Malebary, S. Rahim, and G. Sahar, "Energy efficient tour optimization of wireless mobile chargers for rechargeable sensor networks," *IEEE Systems Journal*, vol. 15, no. 1, pp. 27–36, Mar. 2021.

[4] C. Xu, T. Wu, R. Cheng, and C. W. Yu, "A genetic algorithm for multiple charging car scheduling problem in wireless rechargeable sensor networks," in *Proc. IEEE Asia-Pacific Conf. Antennas Propag. (APCAP)*, 2018, pp. 136–138.

[5] T. T. Huong, P. L. Nguyen, H. T. T. Binh, K. Nguyen, N. M. Hai, and L. T. Vinh, "Genetic algorithm-based periodic charging scheme for energy depletion avoidance in WRSNs," in *Proc. IEEE WCNC*, 2020, pp. 1–6.

[6] M. K. Somesula, R. R. Rout, and D. V. L. N. Somayajulu, "Contact duration-aware cooperative cache placement using genetic algorithm for mobile edge networks," *Computer Networks*, vol. 193, pp. 1389–1286, Apr. 2021.

[7] S. Soni and M. Shrivastava, "Novel wireless charging algorithms to charge mobile wireless sensor network by using reinforcement learning," *SN Applied Sciences*, vol. 1, p. 1052, 2019.

[8] X. Cao, W. Xu, X. Liu, J. Peng, and T. Liu, "A deep reinforcement learning-based on-demand charging algorithm for wireless rechargeable sensor networks," *Ad Hoc Networks*, vol. 110, 2021.

[9] Y. Feng, W. Zhang, G. Han, Y. Kang, and J. Wang, "A newborn particle swarm optimization algorithm for charging scheduling algorithm in industrial rechargeable sensor network," *IEEE Sensors Journal*, vol. 20, no. 18, pp. 1558–1748, Sep. 2020.

[10] Z. Lyu, Z. Wei, X. Wang, Y. Fan, C. Xia, and L. Shi, "A periodic multinode charging and data collection scheme with optimal traveling path in WRSNs," *IEEE Systems Journal*, vol. 14, no. 3, pp. 1937–9234, Sep. 2020.

[11] Y. Dong, S. Li, G. Bao, and C. Wang, "An efficient combined charging strategy for large-scale wireless rechargeable sensor networks," *IEEE Sensors Journal*, vol. 20, no. 17, pp. 1558–1748, Sep. 2020.

[12] Z. Lu, B. Fan, J. Cai, and L. Tang, "J-RCA: A joint routing and charging algorithm with WCE assisted data gathering in wireless rechargeable sensor networks," *IEEE Sensors Journal*, vol. 21, no. 3, pp. 1558–1748, Feb. 2021.

[13] M. Tian, W. Jiao, and G. Shen, "The charging strategy combining with the node sleep mechanism in the wireless rechargeable sensor network," *IEEE Access*, vol. 8, pp. 197817–197827, Nov. 2020.

[14] T. N. Nguyen, B.-H. Liu, S.-I. Chu, D.-T. Do, and T. D. Nguyen, "WRSNs: Toward an efficient scheduling for mobile chargers," *IEEE Sensors Journal*, vol. 20, no. 12, pp. 6753–6761, Jun. 2020.

[15] S. K. Mothku and R. R. Rout, "Fuzzy logic based adaptive duty cycling for sustainability in energy harvesting sensor actor networks," *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 1, pp. 1489–1497, 2022.

[16] H. van Hasselt, "Double Q-learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 23, 2010, pp. 2613–2621.

[17] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[18] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press, 1998.

[19] D. R. Y. Dakey and P. Visvanatharoa, "A genetic based charging scheduling algorithm for network sustainability in wireless rechargeable sensor networks," B.Tech Final Year Project Report, NIT Trichy, 2022.